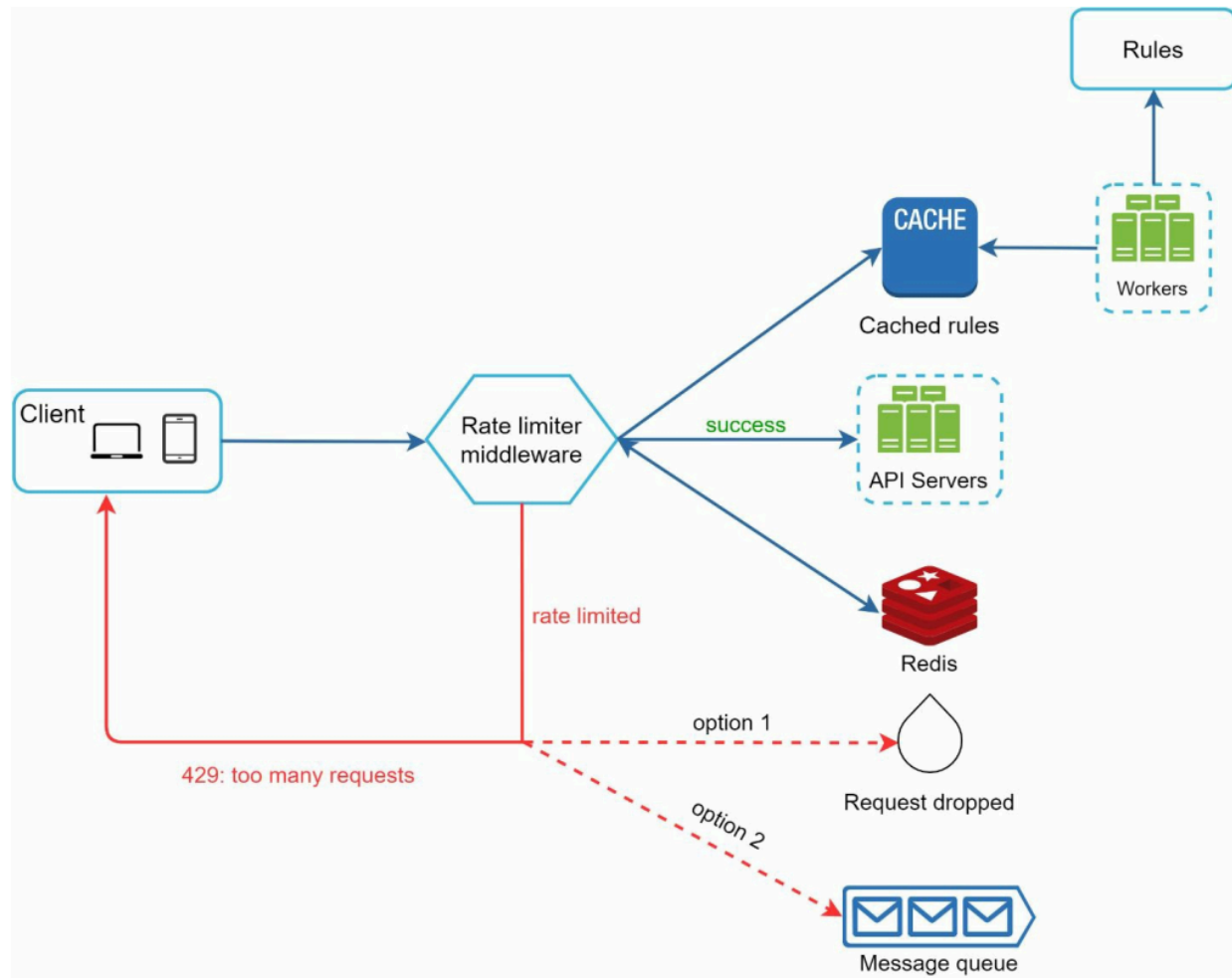


Design a Rate Limiter - Detailed Design



1. System Components Overview

- **Rules (Disk)** → Source of rate limiting policies
- **Workers** → Load rules into cache
- **Cache** → Stores rules for fast access
- **Redis** → Stores counters and timestamps
- **Rate Limiter Middleware** → Core decision-making component
- **API Servers** → Handle valid requests
- **Queue (Optional)** → Handles delayed processing

2. Flow of the System

Step-by-Step Process

1. Rule Management

- Rules are stored on **disk**
- Workers:
 - Periodically fetch rules from disk
 - Store them in **cache**

2. Incoming Request

- Client sends request
- Request first goes to **rate limiter middleware**

3. Rule & Data Fetching

- Middleware loads:
 - Rules from **cache**
 - Counter + last request timestamp from **Redis**

4. Decision Making

Based on fetched data:

Case 1: Request Allowed

- Forward request to **API servers**

Case 2: Request Rate Limited

- Return:

None

HTTP 429 → Too Many Requests

- Additionally:
 - Request is either:
 - **Dropped**, or
 - **Forwarded to queue** for later processing

3. Key Takeaway

Detailed design introduces:

- Separation of **rules storage (disk + cache)**
- Use of **Redis for counters & timestamps**
- Centralized decision via **rate limiter middleware**